

Contents

Slack Support - Intern Implementation Guide	1
Slack support: architecture and intern implementation guide	1
1. Purpose, scope, and how to read this guide	1
2. The system an intern needs to understand	2
3. Slack concepts and the integration gap	5
4. Recommended architecture and decisions	6
5. JavaScript API contract	9
6. Go contracts and runtime ownership	11
7. Ingress, acknowledgment, deduplication, and failures	12
8. Configuration, discovery, and embedding	13
9. Slack app setup and API reference	14
10. Intern implementation plan	16
11. Test plan and acceptance matrix	18
12. What is needed to develop and test	20
13. Review questions and practical limits	21

Slack Support - Intern Implementation Guide

Slack support: architecture and intern implementation guide

1. Purpose, scope, and how to read this guide

This ticket designs a Slack bot host in the discord-bot repository. It is a design deliverable, not an implemented Slack feature. The recommended implementation is independent of the Discord runtime but follows the same concepts: Go owns connectivity, credentials, typed data, validation, and lifecycle; JavaScript supplies bot behavior through a small native module. One selected bot runs in one process.

The user explicitly approved a separate implementation inspired by Discord. Accordingly, this guide does not propose translating Discord handlers, wrapping Discord interfaces, or introducing a universal chat platform abstraction. The existing repository and top-level `go.mod` remain the home for the first implementation. A separate `cmd/slack-bot` binary and Slack packages make the boundary clear without creating another module or repository.

The first milestone is deliberately useful and bounded: an internal, single-workspace Slack app using Socket Mode, a `/golem-ping` command, `app_mention` events, threaded text replies, an offline inspect command, and deterministic tests. Buttons and message updates follow in a second milestone. Modal validation, direct messages, history retrieval, file uploads, public OAuth installation, and multiple installations are later extensions with separate acceptance criteria.

Read sections 2–4 to learn the system, sections 5–9 to understand the proposed contracts, and sections 10–13 to implement and test it. Existing files are identified as **current**; proposed paths and API sketches are **new**. Code blocks labeled pseudocode illustrate contracts and ordering, not compilable implementations.

Evidence baseline

The Discord repository was clean at the beginning of this investigation, at commit 2ea219e6d0a37bb27916dc29a2ba88c1f03b6182. Its `go.mod` declares Go 1.26.4, discordgo v0.29.0, Glazed v1.3.6, and go-go-goja v0.8.3. Slack is not a dependency. Line references below refer to this revision. The adjacent workspace contains newer go-go-goja source; its APIs must not be assumed identical to the pinned dependency.

Official Slack documentation was retrieved on 2026-09-10. The ticket's source inventory includes defuddle Markdown snapshots, original vault notes, source-code snapshots, and hashes. Vault articles explain historical decisions; checked-out code is the authority for current implementation behavior.

2. The system an intern needs to understand

2.1 Go, JavaScript, and the host boundary

Goja is a JavaScript interpreter embedded in a Go process. A bot file can call `require("discord")` because the Go host registers a native module with a CommonJS module loader. This is not a Node.js process: npm packages that require Node networking, timers, filesystem behavior, or native extensions cannot be assumed to work. The Slack SDK should therefore run on the Go side, rather than importing Bolt for JavaScript into Goja.

The go-go-goja engine supplies runtime construction, native module registration, module lookup, an event loop, and runtime ownership. The runtime owner schedules access to the JavaScript VM. Every callback, promise settlement, value conversion, and mutable JavaScript object access must respect that owner. Running two goroutines does not create two safe execution lanes into one VM.

A bot descriptor is metadata collected from a script: its name, commands, events, and runtime configuration schema. A host is the runtime plus the loaded script and its registered handlers. A transport is the external network connection. Keeping these concepts separate lets us inspect a bot without connecting to Slack, test behavior without credentials, and reconnect without reconstructing application state unnecessarily.

2.2 Current Discord startup

The current CLI is built with Cobra and Glazed. Cobra arranges command names and flags; Glazed describes fields, parses settings, and supplies typed values and structured output. The root mounts both direct commands and repository-discovered bot commands. Repository discovery locates candidate scripts, inspection executes their definitions, and a selected bot's schema creates its run command.

`pkg/botcli/command_run.go` decodes Discord settings, validates credentials, collects script configuration, creates the Discord host, optionally syncs commands, opens the session, and waits for cancellation. `internal/bot/bot.go` creates a discordgo session, loads the script, computes intents from event descriptors, binds handlers, and delegates events into the JS

host.

Current Discord path

cmd/discord-bot

|

v

pkg/botcli: discover -> inspect -> build Glazed command

|

v

internal/bot: session + gateway handlers

|

v

internal/jsdiscord.Host

|

v

runtime owner -> JS callback -> typed response -> Discord API

An intent tells Discord which gateway events the session wants. Slack uses app scopes and event subscriptions instead; copying intent inference is not sufficient to configure a Slack installation. Likewise, Discord command bulk overwrite has no direct runtime equivalent in the proposed Slack host.

2.3 Current JavaScript dispatch and response handling

internal/jsdiscord/host.go:21 constructs the engine runtime, registers the Discord and UI modules, permits the database module through middleware, loads the script, and compiles its exported bot. descriptor.go:76 implements inspection by loading a real host, describing it, and closing it. Inspection therefore executes JavaScript; it is not merely parsing a JSON file.

bot_dispatch.go:37 enters JavaScript through bindings.Owner.Call. It builds a per-dispatch context, invokes the handler, and settles the result. Promise polling also returns to the owner to inspect the promise. This is the important idea to reuse, but Slack should have its own response model and asynchronous integration rather than copy a Discord-specific normalizer.

The Discord context includes response functions, a store, config, logging, and Discord operations. Message builders hold state in Go and produce typed results. host_responses.go chooses the correct Discord transport behavior. That division is valuable for Slack too: a Slack text message, an interaction acknowledgment, and a modal validation result are different operations even when all eventually serialize as JSON.

2.4 File reading map

cmd/discord-bot/root.go, newRootCommand

- Why read it: CLI assembly and embedded help
- Slack lesson: Build an independent Slack root with Glazed

pkg/botcli/discover.go, DiscoverBots

- Why read it: Candidate selection and executing inspection
- Slack lesson: Discovery is platform-specific today

pkg/botcli/discover.go, looksLikeBotScript

- Why read it: Literal checks for `require("discord")`
- Slack lesson: Do not expect it to discover Slack scripts

pkg/botcli/model.go:24, DiscoveredBot

- Why read it: Descriptor is a `jsdiscord` type
- Slack lesson: Avoid reusing it as a Slack descriptor

pkg/botcli/run_description.go, addCoreRunFields

- Why read it: Discord credentials and sync flag
- Slack lesson: Define Slack's own host schema

pkg/botcli/runtime_helpers.go:54

- Why read it: Config projection into JS
- Slack lesson: Use an allowlist of bot fields

internal/config/config.go, Settings.Validate

- Why read it: Discord credential validation
- Slack lesson: Slack validation needs two different tokens

internal/bot/bot.go, NewWithScript

- Why read it: Session, host, handlers, lifecycle
- Slack lesson: Keep SDK ownership in Go

internal/jsdiscord/descriptor.go:11

- Why read it: Bot metadata model
- Slack lesson: Name a Slack-specific descriptor

internal/jsdiscord/bot_dispatch.go:37

- Why read it: Owner-thread invocation
- Slack lesson: Never call a handler directly from a socket goroutine

internal/jsdiscord/store.go:11

- Why read it: Namespaced in-memory state
- Slack lesson: A memory store is not durable delivery

pkg/framework/framework.go:59

- Why read it: Public embedding construction
- Slack lesson: Provide a separate Slack embedding API

pkg/xgoja/provider/provider.go:33

- Why read it: Module and command-set provider
- Slack lesson: Treat provider composition as its own integration

internal/jsdiscord/runtime_dispatch_test.go

- Why read it: End-to-end JS dispatch tests

- Slack lesson: Test the host boundary, not just helper functions

2.5 Lessons from go-go-parc

The DSL design article separates composition from domain ownership. It recommends flat functions for simple operations and Go-backed builders when intermediate state has meaningful invariants. For this project that means `ctx.reply({text: ...})` is a reasonable small contract, while a complex Block Kit form builder should have Go-owned state, explicit validation, and a typed built result.

The Discord UI article explains why loose JavaScript UI objects caused late API failures. Slack should validate payloads before network calls, retain typed values once built, and provide errors that identify the exact field or builder method. It should not adopt Discord's row, embed, or response-type vocabulary.

The context article distinguishes runtime lifetime from individual calls. A pending Slack API call belongs to the current invocation and must also stop when the host closes. The xgoja case study shows why module registration, command discovery, selected profiles, and runtime construction are distinct boundaries. Merely registering `require("slack")` will not make a generated runner discover and execute Slack bots.

The framework history records practical lessons in script inspection, dynamic configuration, and preserving useful JavaScript errors. Treat these articles as design rationale; they contain historical paths and examples, not a promise that all APIs exist in the repository's pinned version.

3. Slack concepts and the integration gap

3.1 Two channels of communication

Socket Mode gives the host an outbound WebSocket through which Slack delivers events and interactive payloads. An app-level token, normally beginning with `xapp-`, with `connections:write` is used to establish that connection. A separate bot token, normally beginning with `xoxb-`, authorizes Web API methods. Socket Mode removes the need for a public inbound HTTP endpoint in this development design. See [Socket Mode and connections scope](#).

Receiving an event and posting a message are separate paths. An acknowledgment tells Slack that the envelope was received; it does not mean the bot's business work succeeded. A user-facing reply normally uses a Web API call or an interaction's response URL. This separation allows receipt to remain fast even when JavaScript or downstream services are slow.

3.2 IDs and message references

Slack identifiers must remain strings. A workspace is identified by `team_id`; channels and users have their own IDs. A message is addressed by its channel plus `ts`, Slack's timestamp-shaped identifier. A thread uses the root message's `ts` as `thread_ts`. Do not convert timestamps to floats: precision loss changes identifiers.

For a mention in an existing thread, reply using its `thread_ts`; for a top-level mention, use its `ts` as the reply's thread root. Persist references as `(team_id, channel_id, ts)`, not as `ts` alone. `chat.postMessage` accepts thread targeting and returns a message reference. See [method reference](#).

3.3 Commands and events are configured outside the bot process

A Slack slash command is registered in app settings or an app manifest. Its argument tail is text; Slack does not supply Discord's typed nested slash-option schema. The MVP exposes /golem-ping without inventing a universal argument parser. Future commands may parse their own text or use a separate documented grammar.

The first manifest requests commands, chat:write, and app_mentions:read, and subscribes to app_mention. The app must be installed in the development workspace and invited into the test channel. Direct messages require additional subscription and scope configuration and are not covered by app_mention. See slash commands, mentions, and message.im.

3.4 Acknowledgment and interaction deadlines

Slash commands must be acknowledged within three seconds. The host should aim much lower internally and should never wait for arbitrary JavaScript before acknowledging ordinary commands or events. For the MVP, slash commands are acknowledged empty and their results sent through a response URL; mention events are acknowledged and answered through chat.postMessage.

A response URL is a temporary capability to answer a particular interaction. Keep it in Go, redact it from logs, enforce its lifetime and usage constraints, and never let a bot script supply an arbitrary destination URL. Slack directs applications that need to respond after thirty minutes to use standard message publishing instead. See handling interactions.

Modal submissions are different: returning validation errors or replacing a view requires a payload in the acknowledgment itself. An unconditional early empty acknowledgment would close the modal and destroy the opportunity to return validation errors. This is why modals are a separate phase with a dedicated synchronous validation contract.

4. Recommended architecture and decisions

Decision A: an independent Slack implementation

- **Context:** Discovery, settings, descriptors, operations, and rendering are Discord-specific.
- **Options:** Translate Discord objects; introduce a shared platform interface; implement Slack independently using the same engine concepts.
- **Decision:** Independent Slack packages and cmd/slack-bot, within the existing module.
- **Rationale:** The user explicitly permits this approach. It makes Slack semantics visible and keeps the initial change reviewable.
- **Consequences:** Some discovery and metadata mechanics will resemble Discord. Extract a shared utility only after actual duplicated behavior is proven; no compatibility wrappers or adapters are required.
- **Status:** Proposed implementation design, aligned with user direction.

Decision B: Socket Mode and one installation

- **Context:** Development needs incoming events without deploying an HTTP service.
- **Decision:** Socket Mode, one app installation, one selected bot, one process.
- **Consequences:** Outbound HTTPS and WebSocket access are required. Multiple development processes using the same app can receive different envelopes, so use a dedicated development app and a single active instance. Distributed installations and public OAuth storage are a different project.
- **Status:** Proposed MVP scope.

Decision C: Go owns acknowledgment and side effects

- **Context:** Slack receipt deadlines are shorter than arbitrary bot work.
- **Decision:** A Go ingress loop handles receipt and admission; a separate dispatcher schedules JavaScript; an outbound service performs API work and returns promises.
- **Consequences:** The queue has explicit capacity, timeout, and overload behavior. MVP delivery is best effort after acknowledgment, not durable or exactly once.
- **Status:** Proposed; verify SDK acknowledgment behavior in the first spike.

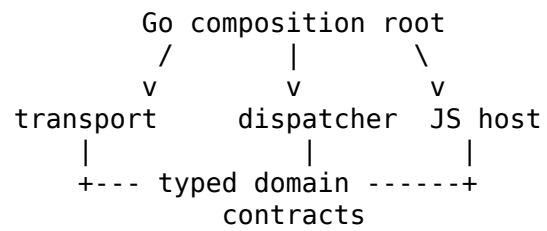
Package layout

All paths below are new. Do not create another `go.mod`.

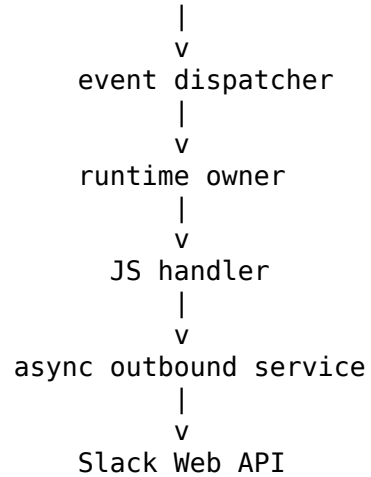
<code>cmd/slack-bot/</code>	entry point and Glazed root
<code>pkg/slackbot/</code>	public settings, host, domain types
<code>host.go</code>	Run/Close and resource ownership
<code>settings.go</code>	explicit configuration validation
<code>events.go</code>	typed inbound snapshots
<code>messages.go</code>	payloads and message references
<code>service.go</code>	outbound operations and policy
<code>internal/slacktransport/</code>	slack-go Web API and Socket Mode
<code>internal/jsslack/</code>	<code>require("slack")</code> , handlers, codecs
<code>pkg/slackcli/</code>	discover, inspect, list, run, manifest
<code>pkg/slackprovider/</code>	later xgoja integration
<code>pkg/doc/topics/</code>	Slack API help page
<code>pkg/doc/tutorials/</code>	Slack setup and authoring tutorial
<code>examples/slack-bots/ping/</code>	first working bot
<code>examples/slack/</code>	development app manifest

Avoid import cycles: `pkg/slackbot` owns domain interfaces and types; `internal/slacktransport` implements those interfaces; the composition root constructs both. `internal/jsslack` consumes domain types and injected service interfaces. A public convenience constructor can be placed in a separate composition package if importing both implementation packages would create a cycle. Do not put an internal transport import into the domain package and then

have the transport import that package back.



Slack socket -> decode/filter -> bounded admission -> ACK



5. JavaScript API contract

5.1 Authoring a bot

This is the target API, not executable with today's binary:

```
const { defineBot } = require("slack");

module.exports = defineBot(({ configure, command, event }) => {
  configure({
    name: "ping",
    description: "Slack transport and runtime smoke bot",
    run: { fields: {
      greeting: { type: "string", default: "pong" }
    }}
  });

  command("/golem-ping", {
    description: "Check the development bot"
  }, async (ctx) => {
    return { text: ctx.config.greeting };
  });

  event("app_mention", async (ctx) => {
    await ctx.reply({ text: "I received your mention." });
  });
});
```

defineBot collects a typed registry. Registration is synchronous and occurs only during loading. Duplicate command names, unsupported event names, invalid configuration schemas, and non-callable handlers fail inspection with a script path and registration name. Keep module initialization declarative: network access and durable writes belong inside handlers, not at import time.

5.2 Context and results

ctx.teamId, channelId, userId

- Contract: Immutable string IDs when present

ctx.command, ctx.text

- Contract: Slash command name and argument text

ctx.event

- Contract: Detached, typed event snapshot; never a live SDK pointer

ctx.config

- Contract: Only fields declared by this bot's schema

ctx.log.info(message, fields)

- Contract: Structured, redacted logging with invocation IDs

ctx.store.get/set/delete/keys

- Contract: Process-local bot/workspace-scoped state

ctx.reply({text})

- Contract: Promise; slash response is ephemeral by default, mention reply is threaded

ctx.slack.messages.post({channelId, text, threadTs})

- Contract: Explicit outbound message; returns {channelId, ts}

ctx.slack.messages.update({channelId, ts, text})

- Contract: Phase 2; update an app-owned message

Expose reply only for contexts that can support it. A connection lifecycle event has no implicit channel and cannot reply. Reject threadTs with a missing target channel in explicit outbound calls. Validate string sizes, required fields, and unknown keys before the API call. The MVP text contract intentionally does not accept arbitrary SDK options or raw Slack JSON.

A handler may either call `ctx.reply` once or return one text response. Returning a response after explicitly replying raises `already_replied`; it does not silently send a duplicate. Returning undefined performs no automatic response. Explicit outbound `messages.post` is a different operation and does not consume the implicit reply slot. The host tracks this state in Go per invocation.

For an ordinary slash response, `ctx.reply` resolves to a delivery receipt such as `{delivered: true, via: "response_url"}`; it must not promise a message timestamp when the transport does not return one. An explicit `messages.post` returns a real message reference. This distinction prevents scripts from trying to edit an ephemeral command response with `chat.update`.

5.3 Error and value conversion

Expose domain errors with stable fields: `code`, `operation`, `message`, and optional `retryAfterSeconds`. Examples include `invalid_argument`, `missing_scope`, `not_in_channel`, `rate_limited`, `context_closed`, and `delivery_unknown`. Never include tokens, response URLs, or full socket connection URLs in errors.

Reject an unknown field with a path, for example `slack.messages.post.threadTS: unknown field`; use `threadTs`. Preserve a useful JavaScript error string and stack while still on the runtime owner; exporting an Error object as a generic map may lose its message. Preserve identifiers as strings, distinguish omitted values from empty values, and reject a builder from another VM rather than accepting a foreign Goja object.

6. Go contracts and runtime ownership

A small domain service should be testable without Goja or the Slack SDK. These proposed interfaces are for Slack implementation boundaries and fakes, not for adapting Discord:

```
// Contract sketch; imports and implementations omitted.
type MessageRef struct {
    ChannelID string
    TS         string
}
type PostMessage struct {
    ChannelID string
    Text      string
    ThreadTS  string
}
type MessageService interface {
    Post(context.Context, PostMessage) (MessageRef, error)
}
type Dispatcher interface {
    Dispatch(context.Context, Invocation) error
}
// In the implementation package:
var _ MessageService = (*WebMessages)(nil)
```

Use github.com/pkg/errors for wrapping at Go boundaries and preserve typed errors for programmatic handling. All network methods take a context. The composition root creates an `errgroup.WithContext` for transport, dispatcher, and managed worker lifecycle. Host shutdown cancels work, stops admission, waits with a bound, then closes the runtime. An idempotent close is required because error unwinding and normal shutdown may meet.

Asynchronous native operation pseudocode

JS calls `ctx.slack.messages.post(input)`, on VM owner:

- validate and decode input into a detached Go value
- capture invocation context and immutable service reference
- allocate promise on this VM
- schedule managed Go worker:
 - `result, error = service.Post(invocationContext, input)`
 - post settlement callback to runtime owner
 - if host already closed: release work; do not touch VM
- return promise immediately

Settlement callback, on VM owner:

- create JS result or JS Error
- resolve or reject promise

Do not perform blocking HTTP inside the owner's callback. Do not hold a Goja object in a worker and call `Export`, `ToValue`, or a promise resolver from that worker. Decode first; return to the owner to encode and settle. A handler awaiting a promise must leave the owner free to run its completion callback, otherwise the system deadlocks.

Use the pinned engine's runtime owner and lifetime services, verifying actual method names before implementation. The adjacent source's `pkg/runtimebridge/runtimebridge.go` documents `Call`, `Post`, lifetime context, and current-call context. That is an orientation reference, not authorization to upgrade dependencies or copy newer APIs blindly.

Invocation lifetime

The process context lasts until shutdown. The bot runtime context lasts until its host closes. Each invocation gets a configured execution deadline and its outbound calls inherit that deadline. An invocation closes after its handler and supported pending operations finish. A retained ctx used later must fail with `context_closed`; it must not silently borrow the credentials or routing data of a newer invocation.

The socket acknowledgment context is separate and short-lived. Canceling it immediately after ACK must not cancel admitted bot work. Conversely, giving bot work `context.Background()` would detach it from shutdown. These two mistakes look similar in simple ping tests and need explicit lifecycle tests.

7. Ingress, acknowledgment, deduplication, and failures

7.1 Admission algorithm

The proposed MVP uses a bounded in-memory queue and a bounded deduplication cache. These bounds are configurable host fields with conservative defaults chosen during the first spike. The ingress path never enters JavaScript.

```
on socket envelope:
  decode payload and envelope identity
  verify expected app/workspace where fields are present
  reject unsupported types and bot/self message events
  derive dedupe key:
    Events API: (team_id, event_id)
    other envelope: envelope_id
  atomically check/reserve key with a TTL
  if duplicate:
    acknowledge this envelope; return
  if queue has capacity:
    admit detached invocation
    acknowledge envelope immediately
    mark reservation admitted
  else:
    remove reservation
    acknowledge with a busy response when supported
    otherwise acknowledge and record an overload drop
```

The proposed overload policy sacrifices delivery rather than creating an unbounded queue or pretending Slack guarantees infinite retries. Log drops and expose a counter. For commands, provide a short ephemeral busy result where envelope response payloads are supported; otherwise use a bounded response-URL error path. Do not block ACK on that error path.

Admission and reservation must be synchronized so two concurrent deliveries cannot both enqueue work. Events API `event_id` identifies the application event; Socket Mode `envelope_id` identifies a delivery envelope. Deduplicating only on the envelope ID may not suppress a re-tried application event with a different envelope. The MVP guarantee for interactions is weaker where no stable event identifier exists; document it and do not invent exactly-once behavior.

There is a crash window after ACK and before completion. Memory admission cannot eliminate it. For workflows that require durable processing, a later phase must persist an inbox before ACK, recover pending work, and journal side effects. Even then, an HTTP timeout after Slack accepted a message leaves uncertain delivery unless a reconciliation strategy exists.

7.2 Filtering and event scope

Validate the configured development workspace and app identity against startup authentication and incoming payloads. Drop messages authored by the bot, bot-message subtypes, and unhandled message changes. For the mention-only MVP there is no subscription to all channel messages. When message events are added, dispatch only the supported subtypes and avoid receiving the same business action through both mention and general message handlers.

Use event-specific snapshots. An `app_mention` snapshot can contain `userId`, `channelId`, `text`, `ts`, and optional `threadTs`; a reaction event has different fields. Do not create a large catch-all map that lets scripts depend on undocumented SDK internals.

7.3 Rate limiting and reconnects

Respect HTTP 429 and `Retry-After` in a cancellable outbound policy, scoped by workspace and method, with channel-aware pacing for message posting. Slack generally allows about one message per second per channel, with additional workspace constraints; do not use tolerated bursts as a throughput target. See rate limits.

Distinguish definitive rejection from uncertain delivery. A 429 can be retried after its delay within the invocation deadline. A connection timeout after sending `chat.postMessage` may mean Slack accepted the message; return `delivery_unknown` instead of automatically duplicating it. Select one retry owner so an SDK retry and a host retry do not multiply.

Let the selected Socket Mode client manage protocol reconnects where supported; inspect its behavior at the pinned version. Keep disconnect handling outside the JavaScript queue, stop on revoked credentials rather than retry forever, and retain in-memory dedupe state across a reconnect within one process. Test shutdown while disconnected as well as shutdown during active requests.

8. Configuration, discovery, and embedding

8.1 Proposed CLI

```
slack-bot bots list --bot-repository ./examples/slack-bots
slack-bot bots help ping --bot-repository ./examples/slack-bots
slack-bot bots ping run --bot-repository ./examples/slack-bots
slack-bot bots ping manifest --bot-repository ./examples/slack-bots
slack-bot validate-config
```

Implement these commands with Glazed fields and sections, following the installed command-authoring conventions when coding begins. Host fields belong in a Slack-specific section; bot fields belong in a separate declared schema. Suggested host fields are `bot-token-file`, `app-token-file`, `team-id`, `app-id`, `allowed-channel`, `handler-timeout`, `queue-capacity`, and `log-level`. Read token files explicitly, trim their surrounding whitespace, and do not introduce direct environment reads. Accept credentials as typed Go values for embedding.

Token-file paths avoid tokens in shell history and process arguments. Restrict files to the operator, keep them outside the repository, and never include their contents in debug dumps. Tests should use synthetic marker strings and verify that neither raw values nor token fragments appear in `ctx.config`, help, structured output, errors, or logs.

The current Discord code uses a blacklist of host-managed fields in `runtime_helpers.go:74`. Slack should instead project only declared bot fields into `ctx.config`. This avoids leaking future credentials when a developer forgets to update a blacklist. Inspection and manifest output require no credentials and must not construct an authenticated SDK client.

8.2 Discovery contract

Keep the Slack discovery implementation separate. Define a documented entry convention: top-level bot `.js` files and `<bot>/index.js`, excluding hidden directories and `node_modules`, with a Slack-specific marker or explicit entry metadata. Do not inspect every helper file. Detect duplicate bot names deterministically and report both paths.

A text marker is a candidate heuristic, not a JavaScript parser. Prefer an explicit entry convention and then execute the definition in a restricted inspection runtime. Inspection grants only registration and safe declared modules, no Slack outbound service. Put a deadline on inspection and test a script that never terminates. A new Slack bot must not require Discord tokens simply because command construction loads metadata.

8.3 Public embedding and provider work

A public embedding API should accept explicit settings, a script path, runtime options, and injected services for tests. `Run(ctx)` owns the long-lived work. The new entry point composes the transport and JS implementation, while domain types remain import-cycle free. Document one complete Go example with signal cancellation and error handling.

Xgoja support is a later integration task: register the Slack module, expose a Slack command-set provider, carry the selected runtime profile and Glazed values into actual runtime construction, and test `inspect/run/generated` command paths. The current Discord provider imports `pkg/xgoja/providerapi`; newer vault material discusses v2 APIs. Choose the contract actually present in the pinned module, or schedule an explicit dependency migration. Do not combine a provider migration with the first Slack ping implementation.

9. Slack app setup and API reference

9.1 Development app manifest

This is a proposed manifest skeleton for the dedicated internal development app. Import it through Slack's app configuration UI, verify it there, and install the app. It is not claimed to

have been installed or validated against an account in this research session.

```
display_information:
  name: Golem Slack Dev
features:
  bot_user:
    display_name: Golem Dev
    always_online: false
  slash_commands:
    - command: /golem-ping
      description: Check the development bot
      should_escape: true
oauth_config:
  scopes:
    bot:
      - commands
      - chat:write
      - app_mentions:read
settings:
  socket_mode_enabled: true
  interactivity:
    is_enabled: true
  event_subscriptions:
    bot_events:
      - app_mention
```

Generate the app-level token separately with `connections:write`; it is not a bot OAuth scope. Install or reinstall after changing OAuth scopes. Invite the bot into the intended test channel. Manifest generation should emit configuration for review, not silently change a live app or imply command synchronization occurred. See manifest reference.

9.2 API reference map

The archived sources are the convenient offline reading copy; the links are the upstream authority.

apps.connections.open

- Purpose in this design: Acquire Socket Mode connection
- Reading: Socket Mode, source 01

auth.test

- Purpose in this design: Validate bot identity and workspace before admission
- Reading: auth.test, source 11

Slash command payload

- Purpose in this design: Command name, argument text, user/channel, response URL
- Reading: commands, source 03

chat.postMessage

- Purpose in this design: Explicit posts and mention thread replies
- Reading: post, source 04

chat.update

- Purpose in this design: Phase 2 updates to app-owned messages
- Reading: update, source 12

views.open

- Purpose in this design: Later modal opening using a trigger ID
- Reading: views.open, source 13

Interaction response

- Purpose in this design: Receipt, ephemeral response, response URL lifetime
- Reading: interactions, source 05

Request signatures

- Purpose in this design: Later HTTP ingress only
- Reading: verification, source 09

Go client

- Purpose in this design: Candidate Web API and Socket Mode implementation
- Reading: slack-go/slack, source 10

The Go client is a community-maintained Go implementation; it is not Slack's Bolt runtime. Pin a reviewed version in the existing module, then confirm its context-aware methods, fake HTTP support, Socket Mode ACK calls, reconnect behavior, and error types. SDK method signatures in this guide are intentionally domain pseudocode until that version is chosen.

9.3 Later capability boundaries

Buttons should use Slack `action_id` routing and native Block Kit structures. Implement Go-owned payload types and validate action IDs, required text, and supported block types. Do not expose Discord `customId`, embeds, rows, or guild roles. `chat.update` cannot update ephemeral messages, so keep ephemeral response handling distinct from persistent message references. See `chat.update`.

Opening a modal uses a short-lived `trigger_id`; do not queue it behind unbounded work. Modal submission errors require an acknowledgment payload keyed by input block IDs. Implement local, bounded validation separately from asynchronous business work. See `views.open` and interaction handling.

If HTTP delivery is added later, verify Slack signatures against the raw request body, reject stale timestamps, compare signatures in constant time, and handle challenge requests. Socket Mode does not require this HTTP signing path. See request verification.

10. Intern implementation plan

Each phase has a concrete output and a completion test. Finish the MVP before expanding the platform surface.

Phase 0: establish the baseline and dependency contract

Read the current file map and the archived DSL/context notes. Run the repository tests and record the existing revision and failures separately from new work. Confirm the active Go workspace and actual go-go-goja version; adjacent checkout source is not enough. Inspect a pinned slack-go release and implement a tiny fake-backed Go test showing context cancellation, ACK invocation, and message serialization.

Done when: a short dependency note records the exact version and SDK methods, and the fake test proves the required transport seams without credentials. Add the dependency only to the existing module.

Phase 1: typed domain and offline script inspection

Create Slack domain settings, events, replies, references, errors, and service interfaces. Build internal/jsslack registration and the descriptor. Add examples/slack-bots/ping/index.js. Implement a restricted inspection runtime and deterministic candidate discovery. Provide list/help output through pkg/slackcli.

Done when: listing and inspecting the example succeeds offline; duplicate registrations, invalid schemas, unsupported event names, and an infinite inspection script fail clearly and release their runtime. No Discord credential or Slack token is needed.

Phase 2: dispatch and asynchronous responses with fakes

Implement invocation-scoped context construction, reply state, Go-owned payload validation, promise-based outbound calls, owner-thread settlement, and bounded cancellation. Keep the transport fake; exercise a real JavaScript handler through the host and assert captured outbound requests.

Done when: a slash handler returns one response, a mention handler posts to the correct thread, rejected promises preserve messages, and a slow outbound operation does not stop unrelated ingress acknowledgment.

Phase 3: Socket Mode and live MVP

Implement settings decoding, explicit token-file loading, startup auth.test, app/workspace checks, the Socket Mode receive loop, bounded admission, dedupe, and lifecycle coordination with errgroup. Wire cmd/slack-bot, add a reviewed manifest, and implement reconnect and redacted logging.

Done when: authorized testing in the dedicated channel demonstrates /golem-ping, a top-level mention, a threaded mention, reconnect recovery, and clean cancellation. Record only IDs and sanitized outcomes, not credentials or private channel transcripts.

Phase 4: operational hardening and intern documentation

Add overload behavior, rate-limit scheduling, cancellation during backoff, unknown-delivery errors, and failure counters. Write embedded help, TypeScript declarations for the JS surface, the manifest command, a token-file setup guide, and one embedding example. Keep generated declarations and examples aligned with the actual runtime contract.

Done when: the negative test matrix passes, help distinguishes current capabilities from later features, and another developer can run the bot from the guide without guessing flag names or token scopes.

Phase 5: buttons and message updates

Add a small Go-backed Block Kit API and `action(actionId, handler)` registration. Route actions to immutable snapshots, acknowledge promptly, and support explicit updates to persistent app messages. Start with a counter or pager example and authorize access using stable user/workspace IDs when behavior is restricted.

Done when: a button changes the original persistent message; an ephemeral response is rejected as an update target; invalid blocks fail before network access. Keep modals and history APIs out of this phase.

Phase 6: optional integrations

Implement xgoja provider support, direct messages, modal validation, persistence, or history access only as separately scoped work. A production durable inbox and multi-installation OAuth storage need their own design because they change delivery, tenancy, and credential ownership.

11. Test plan and acceptance matrix

Most implementation work requires no real tokens. Unit tests exercise codecs and settings; host tests execute actual scripts with fake services; transport tests use local HTTP/WebSocket fixtures; live tests prove the Slack app configuration and network contract.

Offline discovery

- Stimulus: Example repository, no credentials
- Required assertion: Stable name/commands/config; no network calls

Credential isolation

- Stimulus: Distinct synthetic token markers
- Required assertion: Markers absent from JS config, descriptors, logs, errors

Slash reply

- Stimulus: Handler returns text
- Required assertion: One ephemeral response through the captured response capability

Duplicate response

- Stimulus: Explicit reply followed by return
- Required assertion: `already_replied`, no second send

Thread selection

- Stimulus: Top-level and threaded mentions
- Required assertion: Root `ts` and existing `thread_ts` chosen correctly

Precision

- Stimulus: Long fractional timestamp
- Required assertion: Exact string survives decode, dispatch, send

ACK latency

- Stimulus: Handler blocked on fake API
- Required assertion: Receipt recorded before handler completes

Queue saturation

- Stimulus: Fill bounded queue
- Required assertion: Bounded memory, explicit drop/busy metric, no hang

Duplicate event

- Stimulus: Same event ID, different envelope IDs
- Required assertion: Both receipts acknowledged; one handler run

Malformed or foreign event

- Stimulus: Wrong workspace/app or invalid payload
- Required assertion: No handler or outbound side effect

Bot loop

- Stimulus: Self-authored event
- Required assertion: Ignored

JavaScript rejection

- Stimulus: Throw Error; reject promise
- Required assertion: Useful message and stack retained

Promise ownership

- Stimulus: Background API completion
- Required assertion: Settlement runs on owner; race test clean

Shutdown

- Stimulus: Cancel while request/backoff pending
- Required assertion: Workers exit, pending operations fail, runtime closes

Infinite handler

- Stimulus: CPU-bound loop
- Required assertion: Deadline interrupts or host fails boundedly; ACK remains independent

Rate limit

- Stimulus: Fake 429 and Retry-After
- Required assertion: Delay honored; cancellation interrupts wait

Unknown delivery

- Stimulus: Transport timeout after write
- Required assertion: No blind duplicate post; typed uncertain result

Reconnect

- Stimulus: Drop and reopen socket fixture

- Required assertion: Receiver resumes without creating a second host

Inspection safety

- Stimulus: Script tries outbound call on import
- Required assertion: Clear unavailable-capability error

During implementation, run focused tests first, then `go test ./...`, `go build ./...`, and repository lint. Use `go test -race` on the new host, transport, and JS packages because shared runtime ownership is central to this feature. Do not claim baseline failures are caused by Slack; record them before changes.

For live processes use `tmux` and `go run ./cmd/slack-bot ...`, not a build-and-execute loop. Capture the pane for sanitized diagnostic evidence and stop the session after testing. Socket Mode needs no local inbound server. If a later HTTP test server uses a port, follow the workspace's `lsof -who -p PORT -k` cleanup convention.

Live test sequence

1. Confirm the development app, workspace ID, bot identity, and permitted channel with the operator. Obtain explicit authorization to send test messages there.
2. Start the single test process in `tmux` using token-file paths and the configured workspace/channel restriction.
3. Ask a human tester to invoke `/golem-ping`; verify one ephemeral result. A bot token alone does not impersonate a human slash-command invocation.
4. Have the tester mention the app at the top level and inside a thread. Verify the destination references and a single response in each case.
5. With authorized direct posting, perform one Web API smoke message and record its message reference. Delete only if that cleanup was included in the authorization.
6. Interrupt/reconnect the socket and repeat one command; verify graceful shutdown and no remaining process.

12. What is needed to develop and test

Required for the first live milestone

- A dedicated Slack app installed in a development workspace, with Socket Mode enabled.
- A bot OAuth token (`xoxb-...`) with `commands`, `chat:write`, and `app_mentions:read`.
- An app-level token (`xapp-...`) with `connections:write`.
- The workspace/team ID, app ID, and a test channel ID; invite the bot into that channel.
- Event subscription `app_mention` and the `/golem-ping` slash command configured from the manifest.
- Readable, private local token files, with their paths supplied to the developer. Do not place token values in ticket documents or chat transcripts.
- Outbound HTTPS/WebSocket network access and permission to fetch the chosen Go dependencies.
- Explicit permission to send bot test messages in the named channel, plus a human tester for slash commands and later buttons/modals.

The tokens have different jobs and neither replaces the other. A signing secret, OAuth client secret, user token, and public tunnel are not needed for this Socket Mode MVP. A signing secret becomes relevant if HTTP ingress is implemented; client credentials and installation storage become relevant for a distributed OAuth app. An LLM API key is only needed if a later bot actually calls an LLM service.

For direct-message support, add the relevant `message.im` subscription and `im:history` scope for receipt, and `im:write` if opening DMs through the Conversations API. For reading channel history, replies, or member information, select scopes from the specific method references at implementation time rather than requesting broad permissions preemptively.

What can proceed without secrets

All domain modeling, JS API design, registration, inspection, CLI work, fakes, payload validation, cancellation tests, and manifest generation can be implemented offline. Real credentials are needed to verify installation, scope grants, SDK connectivity to Slack, and user interaction behavior. The research session did not read environment secrets, connect a Slack bot, or send messages.

13. Review questions and practical limits

The recommended defaults are an independent binary in the same module, Socket Mode, a single internal installation, and a text-first MVP. The remaining product questions are which real bot workflow should follow ping, whether direct messages are needed immediately after the MVP, and whether workflow reliability requires durable processing. These questions do not block the offline implementation phases.

Before accepting the implementation, review three boundaries closely: owner-thread promise settlement, ACK/admission ordering, and credential projection. Simple success tests often miss errors in these areas. Also verify the chosen SDK version's reconnect and retry behavior so the host does not promise stronger guarantees than its transport provides.

This guide deliberately leaves the implementation ticket active. Delivery of the design and its reMarkable copy completes the documentation request; it does not mean Slack support has shipped.